LangNation AWS Cloud Platform

Simple Guide to Every Main Tool & AWS Service Used So Far

Milestones 1–6

Author: SERAJ AHMAD SIDDIQUI
AWS Cloud Computing with AI Bootcamp
Current scope: Milestones 1–6 + automated sandbox recovery
Region: us-east-1

Purpose of this guide
This document explains the LangNation cloud architecture in very simple language. For every tool or AWS service, it tells you the full form, what it does, why we used it, and an easy real-life analogy. It is designed for revision before a presentation or interview.

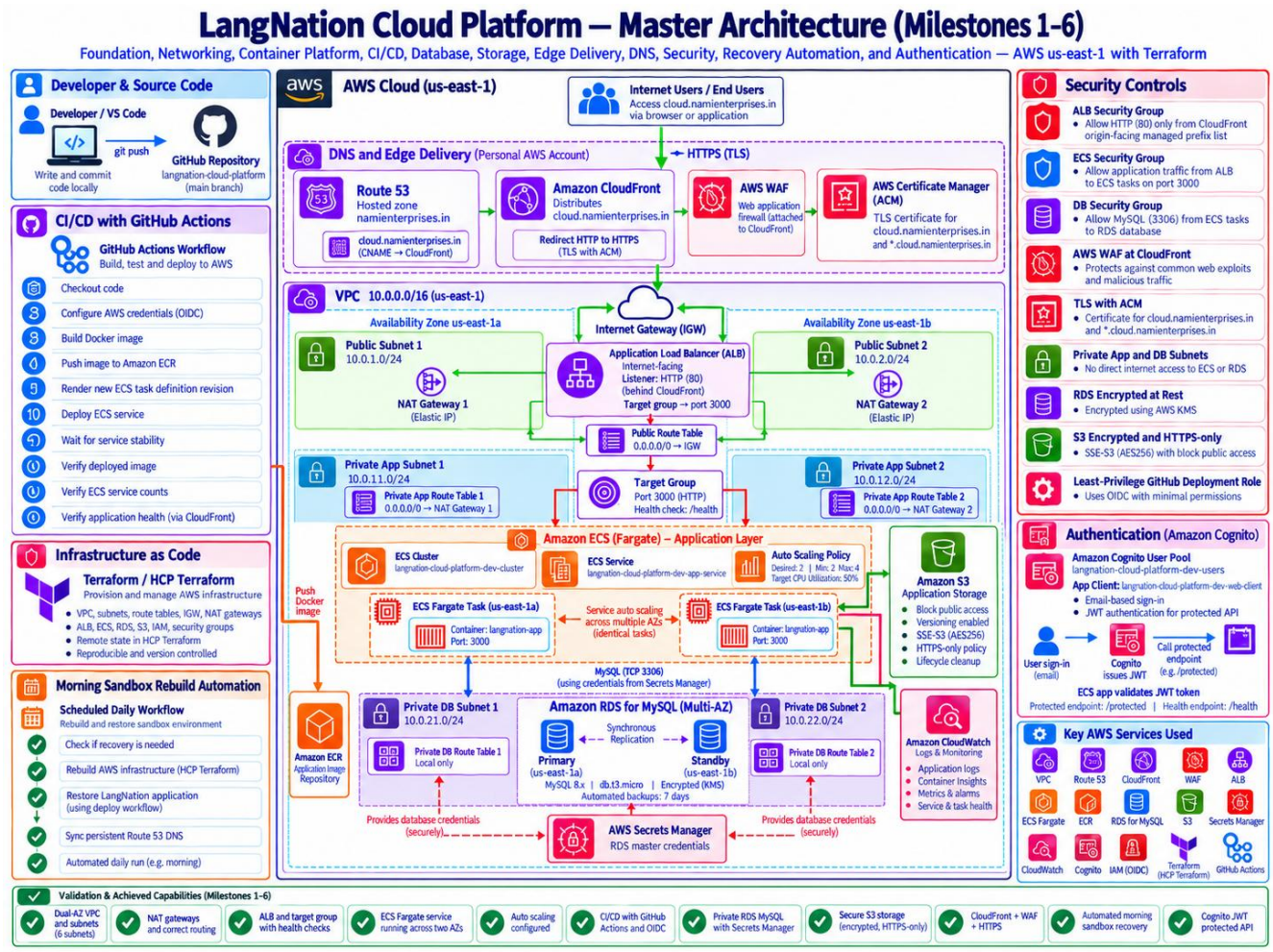
Project endpoint: cloud.namienterprises.in

AWS Region: us-east-1

1. First understand the big picture

One-sentence architecture

A user reaches LangNation through Route 53 → CloudFront → WAF → ALB → ECS Fargate. The application stores data in RDS and S3, uses Cognito for login, CloudWatch for logs, Terraform/HCP Terraform for infrastructure, and GitHub Actions for deployment and morning recovery.



Master architecture implemented through Milestone 6

A very simple request flow

1. User enters https://cloud.namienterprises.in
2. Route 53 finds where the domain should go.
3. CloudFront receives the HTTPS request at the edge.
4. AWS WAF checks whether the request looks dangerous.
5. CloudFront sends allowed traffic to the ALB.
6. ALB forwards the request to healthy ECS Fargate tasks.
7. The Node.js application processes the request.
8. If needed, the app uses Cognito, RDS, S3, Secrets Manager and CloudWatch.

2. Quick-reference table

Area	Tool / Service	Full form	Main job
Development	VS Code	Visual Studio Code	Write and edit project code
Source control	Git / GitHub	Git + GitHub	Track code and store the repository
Automation	GitHub Actions	CI/CD workflow platform	Build, deploy, verify and recover
Containers	Docker	Container platform	Package the Node.js app
Infrastructure	Terraform	Infrastructure as Code tool	Define AWS resources in code
Terraform cloud	HCP Terraform	HashiCorp Cloud Platform Terraform	Remote plans, applies and state
Command line	AWS CLI	Amazon Web Services Command Line Interface	Operate and verify AWS
Identity	IAM	Identity and Access Management	Permissions and roles
Identity federation	OIDC	OpenID Connect	Temporary GitHub-to-AWS authentication
Networking	VPC	Virtual Private Cloud	Private AWS network
Networking	AZ	Availability Zone	Separate data-center location
Networking	Subnet	Subnet	Divide VPC into public/app/database areas
Networking	IGW	Internet Gateway	Internet path for public resources
Networking	Route Table	Route Table	Controls where network traffic goes
Networking	NAT Gateway	Network Address Translation Gateway	Private outbound Internet access
Networking	EIP	Elastic IP	Stable public IP for NAT Gateway
Security	Security Group	Security Group	Stateful firewall around AWS resources
Registry	ECR	Elastic Container Registry	Store Docker images
Containers	ECS	Elastic Container Service	Run and manage containers
Containers	Fargate	AWS Fargate	Serverless compute for ECS
Containers	Task Definition	ECS Task Definition	Blueprint for a container task
Containers	ECS Service	Elastic Container Service Service	Keep tasks running
Traffic	ALB	Application Load Balancer	Distribute HTTP traffic
Traffic	Target Group	Target Group	List of healthy ECS targets
Scaling	Auto Scaling	Application Auto Scaling	Scale ECS tasks automatically
Monitoring	CloudWatch	Amazon CloudWatch	Logs, metrics and monitoring
Database	RDS	Relational Database Service	Managed MySQL database
Availability	Multi-AZ	Multiple Availability Zones	Database resilience
Secrets	Secrets Manager	AWS Secrets Manager	Store RDS credentials safely
Storage	S3	Simple Storage Service	Private object/file storage
DNS	Route 53	Amazon Route 53	Authoritative DNS
TLS	ACM	AWS Certificate Manager	HTTPS certificate
Edge	CloudFront	Amazon CloudFront	Global CDN and HTTPS front door
Web security	WAF	Web Application Firewall	Block malicious web requests
Authentication	Cognito	Amazon Cognito	User sign-in and token issuance
Authorization	JWT	JSON Web Token	Prove an API caller is authenticated

3. Development, Infrastructure as Code & Automation

VS Code

Full form
Visual Studio Code

What it is / does: A code editor used to write Terraform, JavaScript, Docker and workflow files.

Why we used it: It gave us one place to build and manage the LangNation project locally.

Easy way to remember: Like the desk where the engineer writes and organizes all project work.

In our LangNation project: We edited Terraform modules, Node.js application code and GitHub Actions YAML here.

Git

Full form
Git

What it is / does: A version-control system that records changes to files over time.

Why we used it: It lets us track every infrastructure and application change and safely commit only intended files.

Easy way to remember: Like a detailed history book with a restore point for every important change.

In our LangNation project: We used commits for networking, ECS, CI/CD, database, edge security and authentication changes.

GitHub

Full form
GitHub

What it is / does: A hosted platform for Git repositories, collaboration and automation.

Why we used it: It is the central source repository for LangNation and the trigger point for deployments.

Easy way to remember: Like the shared central library that holds the official project copy.

In our LangNation project: Repository: langnation-cloud-platform; the main branch triggers selected workflows.

GitHub Actions

Full form
GitHub Actions (CI/CD automation)

What it is / does: GitHub's workflow engine. It runs automated jobs when code is pushed or on a schedule.

Why we used it: We use it to build Docker images, push them to ECR, deploy ECS, verify health and run morning recovery.

Easy way to remember: Like a robot engineer that follows the same deployment checklist every time.

In our LangNation project: It uses OIDC, builds the application, deploys the current Git SHA and verifies CloudFront health.

Docker

Full form
Docker

What it is / does: Packages the application and its dependencies into a portable container image.

Why we used it: It makes the Node.js application run consistently in development and on ECS Fargate.

Easy way to remember: Like putting the app, runtime and instructions into one sealed shipping box.

In our LangNation project: GitHub Actions builds the Docker image from application/Dockerfile.

Terraform

Full form
Terraform (Infrastructure as Code)

What it is / does: A tool that describes cloud infrastructure using code instead of manual console clicks.

Why we used it: Our sandbox is frequently cleaned, so Terraform lets us reproduce the whole environment reliably.

Easy way to remember: Like an architectural blueprint that can rebuild the same building again and again.

In our LangNation project: Modules create networking, security, ECS, ALB, RDS, S3, CloudFront, WAF and Cognito.

HCP Terraform

Full form
HashiCorp Cloud Platform Terraform

What it is / does: A managed remote Terraform service for plans, applies and state storage.

Why we used it: It is our authoritative Terraform execution environment and keeps remote state safe.

Easy way to remember: Like the project control room that stores the blueprint history and approves construction.

In our LangNation project: Git pushes trigger plans; approved or automated runs apply infrastructure in the sandbox.

AWS CLI

Full form
Amazon Web Services Command Line Interface

What it is / does: A command-line tool for querying and controlling AWS services.

Why we used it: We used it to verify actual AWS state instead of trusting only the console.

Easy way to remember: Like a direct command terminal for speaking to AWS.

In our LangNation project: Used to inspect ECS, ALB target health, ACM, CloudFront, WAF, Cognito, security groups and more.

IAM

Full form
Identity and Access Management

What it is / does: AWS service for users, roles and permissions.

Why we used it: It controls exactly what GitHub Actions and AWS services are allowed to do.

Easy way to remember: Like an access-control system that decides who has which keys.

In our LangNation project: We created least-privilege deployment roles and an ECS task execution role.

OIDC

Full form
OpenID Connect

What it is / does: A standard that lets GitHub prove its identity to AWS and receive temporary credentials.

Why we used it: It avoids storing long-lived AWS access keys in GitHub.

Easy way to remember: Like showing a trusted temporary digital ID instead of carrying a permanent master key.

In our LangNation project: GitHub assumes one sandbox deployment role and one restricted personal-account Route 53 role.

4. Networking & Network Security

AWS Region

Full form
Amazon Web Services Region

What it is / does: A geographic AWS area containing multiple Availability Zones.

Why we used it: We deploy the sandbox resources in us-east-1.

Easy way to remember: Like choosing the city where your cloud infrastructure lives.

In our LangNation project: The main AWS environment is in us-east-1.

Availability Zone (AZ)

Full form
Availability Zone

What it is / does: An isolated data-center location inside an AWS Region.

Why we used it: Using two AZs makes the platform more available if one location has a problem.

Easy way to remember: Like operating from two separate buildings in the same city.

In our LangNation project: We use us-east-1a and us-east-1b.

VPC

Full form
Virtual Private Cloud

What it is / does: A logically isolated private network inside AWS.

Why we used it: It gives LangNation its own controlled network for public, application and database resources.

Easy way to remember: Like a private campus with its own roads, entrances and security rules.

In our LangNation project: CIDR: 10.0.0.0/16.

Subnet

Full form
Subnet

What it is / does: A smaller network segment inside a VPC.

Why we used it: We separate public resources, private application tasks and private databases for security.

Easy way to remember: Like dividing a campus into public reception, staff offices and a locked records room.

In our LangNation project: Public: 10.0.1.0/24 & 10.0.2.0/24; App: 10.0.11.0/24 & 10.0.12.0/24; DB: 10.0.21.0/24 & 10.0.22.0/24.

Internet Gateway (IGW)

Full form
Internet Gateway

What it is / does: Connects a VPC's public subnets to the Internet.

Why we used it: The public ALB/NAT layer needs a controlled path to and from the Internet.

Easy way to remember: Like the main front gate between the private campus and the public road.

In our LangNation project: The public route table sends 0.0.0.0/0 to the IGW.

Route Table

Full form
Route Table

What it is / does: Contains network routes that decide where packets should go.

Why we used it: It gives public and private subnets different network behavior.

Easy way to remember: Like road signs that tell traffic which exit to use.

In our LangNation project: Public → IGW; private app → same-AZ NAT; private DB → local only.

NAT Gateway

Full form
Network Address Translation Gateway

What it is / does: Allows private resources to start outbound Internet connections without becoming publicly reachable.

Why we used it: ECS tasks in private subnets can download dependencies or contact AWS/public endpoints safely.

Easy way to remember: Like a one-way exit gate: staff can go out, but strangers cannot directly walk in.

In our LangNation project: One NAT Gateway per AZ for the private application subnets.

Elastic IP (EIP)

Full form
Elastic IP Address

What it is / does: A static public IPv4 address allocated by AWS.

Why we used it: Each NAT Gateway needs a stable public address.

Easy way to remember: Like a fixed street number for the outbound gateway.

In our LangNation project: Two EIPs, one for each NAT Gateway.

Security Group

Full form
Security Group

What it is / does: A stateful virtual firewall attached to AWS resources.

Why we used it: It enforces least-privilege network access between CloudFront, ALB, ECS and RDS.

Easy way to remember: Like a security guard with a list of exactly who may enter and on which door.

In our LangNation project: ALB: CloudFront-only port 80; ECS: port 3000 from ALB only; DB: MySQL 3306 from ECS only.

5. Containers, Load Balancing, Scaling & Monitoring

ECR

Full form
Elastic Container Registry

What it is / does: A private AWS registry that stores Docker container images.

Why we used it: GitHub Actions needs a secure place to push versioned LangNation images before ECS deploys them.

Easy way to remember: Like a warehouse containing approved application packages.

In our LangNation project: Repository: langnation-cloud-platform-dev-app; images are tagged with the Git commit SHA.

ECS

Full form
Elastic Container Service

What it is / does: AWS service for running and managing containers.

Why we used it: It manages our Node.js containers without us manually operating individual servers.

Easy way to remember: Like a manager that assigns work to container workers and keeps them running.

In our LangNation project: Cluster: langnation-cloud-platform-dev-cluster.

AWS Fargate

Full form
AWS Fargate

What it is / does: Serverless compute for containers. AWS manages the underlying servers.

Why we used it: We want to run ECS containers without provisioning or patching EC2 instances.

Easy way to remember: Like renting ready-to-use workstations without owning or maintaining the hardware.

In our LangNation project: Each task uses 256 CPU units and 512 MiB memory in private app subnets.

ECS Task Definition

Full form
Elastic Container Service Task Definition

What it is / does: A versioned blueprint describing the container image, ports, resources, logs and environment variables.

Why we used it: Every deployment creates a reproducible definition of exactly how the app should run.

Easy way to remember: Like a recipe card for starting one application worker.

In our LangNation project: Defines langnation-app, port 3000, CloudWatch logging and Cognito environment variables.

ECS Service

Full form
Elastic Container Service Service

What it is / does: Maintains the desired number of ECS tasks and replaces failed tasks.

Why we used it: It provides self-healing and stable application capacity.

Easy way to remember: Like a supervisor who always makes sure enough workers are on duty.

In our LangNation project: Baseline desired count is 2 tasks across two AZs.

ALB

Full form
Application Load Balancer

What it is / does: Layer-7 load balancer that receives HTTP requests and distributes them to healthy targets.

Why we used it: It gives ECS a stable entry point and balances traffic between tasks in multiple AZs.

Easy way to remember: Like a receptionist who sends each visitor to an available employee.

In our LangNation project: CloudFront is the only allowed ingress source; ALB forwards to target group on port 3000.

Target Group

Full form
Application Load Balancer Target Group

What it is / does: A group of backend targets that the ALB can send requests to.

Why we used it: It connects the ALB to ECS task IPs and continuously checks their health.

Easy way to remember: Like the receptionist's live list of employees who are currently available.

In our LangNation project: Target type IP; port 3000; health check path /health.

Service Auto Scaling

Full form
Application Auto Scaling for ECS

What it is / does: Automatically changes the number of ECS tasks based on load.

Why we used it: It allows the application to handle higher traffic while keeping a controlled minimum and maximum.

Easy way to remember: Like calling extra staff when the queue gets busy and sending them home when demand falls.

In our LangNation project: Minimum 2, maximum 4, target average CPU 50%.

CloudWatch

Full form
Amazon CloudWatch

What it is / does: AWS monitoring service for logs, metrics and operational visibility.

Why we used it: We need logs from ECS and monitoring data to troubleshoot and prove the system is healthy.

Easy way to remember: Like CCTV, dashboards and a logbook for the cloud platform.

In our LangNation project: ECS application log group uses 7-day retention and ECS Container Insights is enabled.

6. Database, Secrets & File Storage

RDS

Full form
Relational Database Service

What it is / does: A managed AWS service for relational databases such as MySQL.

Why we used it: We need a reliable private database without managing database servers ourselves.

Easy way to remember: Like renting a professionally managed records system instead of building the database server yourself.

In our LangNation project: MySQL 8.4.x, db.t3.micro, private DB subnets, encrypted storage, 7-day backups.

Multi-AZ

Full form
Multiple Availability Zones

What it is / does: RDS keeps a synchronous standby database in another Availability Zone.

Why we used it: It improves database availability if the primary AZ or database instance has a problem.

Easy way to remember: Like keeping a synchronized backup office in another building ready to take over.

In our LangNation project: Primary and standby span us-east-1a / us-east-1b.

Secrets Manager

Full form
AWS Secrets Manager

What it is / does: Securely stores sensitive credentials and secrets.

Why we used it: Database passwords should not be hard-coded in Terraform or application code.

Easy way to remember: Like a locked vault that only authorized services can open.

In our LangNation project: RDS manages the master user password through Secrets Manager.

S3

Full form
Simple Storage Service

What it is / does: Highly durable object storage for files and application data.

Why we used it: It gives LangNation private storage for future uploads and application objects.

Easy way to remember: Like a secure cloud filing cabinet for files rather than database rows.

In our LangNation project: Private bucket, Block Public Access, versioning, SSE-S3 AES256 encryption and HTTPS-only policy.

7. DNS, HTTPS, Edge Security & Authentication

DNS

Full form
Domain Name System

What it is / does: The Internet naming system that translates human-friendly names into service destinations.

Why we used it: Users should access a stable domain instead of remembering changing AWS addresses.

Easy way to remember: Like the Internet's phone book.

In our LangNation project: cloud.namienterprises.in is the public cloud endpoint.

Route 53

Full form
Amazon Route 53

What it is / does: AWS authoritative DNS service.

Why we used it: We use it for reliable DNS and to point cloud.namienterprises.in to the current CloudFront distribution.

Easy way to remember: Like the official address directory for the domain.

In our LangNation project: The hosted zone lives in the personal AWS account so DNS survives nightly sandbox cleanup.

ACM

Full form
AWS Certificate Manager

What it is / does: Creates and manages TLS certificates used for HTTPS.

Why we used it: CloudFront needs a trusted certificate for cloud.namienterprises.in.

Easy way to remember: Like the verified digital certificate that creates the browser padlock.

In our LangNation project: Certificate covers cloud.namienterprises.in and *.cloud.namienterprises.in; DNS validation persists in Route 53.

TLS / HTTPS

Full form
Transport Layer Security / Hypertext Transfer Protocol Secure

What it is / does: Encrypts traffic between the user's browser and the application front door.

Why we used it: Credentials and API traffic must not travel as plain text.

Easy way to remember: Like putting messages inside a locked envelope while they travel.

In our LangNation project: CloudFront redirects HTTP to HTTPS and uses the ACM certificate.

CloudFront

Full form
Amazon CloudFront

What it is / does: AWS global Content Delivery Network and edge front door.

Why we used it: It provides HTTPS, a stable edge endpoint and the layer where WAF protects the application.

Easy way to remember: Like a global reception desk close to users that forwards approved requests to the origin.

In our LangNation project: Custom alias cloud.namienterprises.in; caching disabled for the current dynamic API.

WAF

Full form
Web Application Firewall

What it is / does: Inspects HTTP requests and blocks patterns associated with web attacks or abusive traffic.

Why we used it: It adds an application-security layer before requests reach the ALB and ECS.

Easy way to remember: Like a security scanner at the entrance that checks every visitor's request.

In our LangNation project: AWS Managed Common Rule Set + per-IP rate rule; controlled XSS test returned HTTP 403.

Cognito

Full form
Amazon Cognito

What it is / does: Managed user authentication service for sign-up, sign-in and token issuance.

Why we used it: We need a secure login system without building password storage and token issuance ourselves.

Easy way to remember: Like the identity desk that checks a user's login and issues a digital access pass.

In our LangNation project: Email-based User Pool and public web App Client; /protected requires a valid access token.

JWT

Full form
JSON Web Token

What it is / does: A signed token containing claims that can prove a user has authenticated.

Why we used it: The Express API can validate the Cognito access token before allowing protected requests.

Easy way to remember: Like a signed temporary digital ID card.

In our LangNation project: No token → /protected returns 401; valid Cognito access token → 200.

8. Important design decisions to remember

Why private Route 53?
The neue-fische sandbox is disposable and can be cleaned overnight. The personal AWS Route 53 hosted zone is persistent, so the domain, mail DNS and ACM validation survive. The morning workflow updates only the cloud subdomain to the new CloudFront address.

Why ECS and RDS are private
They should not accept direct Internet traffic. Users go through CloudFront/WAF/ALB; the database accepts MySQL only from ECS.

Why ALB direct access is blocked
Without this restriction, someone could bypass CloudFront and WAF by calling the ALB DNS directly. The ALB security group now allows port 80 only from the AWS-managed CloudFront origin-facing prefix list.

Why OIDC instead of AWS keys
GitHub receives temporary AWS credentials only when the workflow runs. There are no permanent AWS access keys stored in GitHub.

Why Terraform + HCP Terraform
Terraform makes the environment reproducible; HCP Terraform keeps the authoritative remote state and controlled plan/apply workflow.

Why Git SHA image tags
Every deployed container image can be traced back to the exact Git commit that created it.

Why /health stays public
Load balancers, CloudFront and deployment checks need a simple endpoint to prove the service is alive. Sensitive application endpoints can require Cognito JWTs.

9. Our most important network numbers

Item	Value	Meaning
VPC	10.0.0.0/16	Whole LangNation AWS network
Public subnet 1	10.0.1.0/24	AZ us-east-1a
Public subnet 2	10.0.2.0/24	AZ us-east-1b
Private app subnet 1	10.0.11.0/24	ECS tasks in AZ 1
Private app subnet 2	10.0.12.0/24	ECS tasks in AZ 2
Private DB subnet 1	10.0.21.0/24	RDS DB subnet AZ 1
Private DB subnet 2	10.0.22.0/24	RDS DB subnet AZ 2
ALB → ECS	TCP 3000	Application traffic
ECS → RDS	TCP 3306	MySQL traffic
Public domain	cloud.namienterprises.in	User-facing HTTPS endpoint
/health	HTTP 200	Public health check
/protected	401 without JWT / 200 with JWT	Authenticated API example

10. Two flows you should be able to explain in a presentation

A. User request flow
User → Route 53 → CloudFront (HTTPS) → WAF → ALB → Target Group → ECS Fargate → Node.js API. If the endpoint is protected, the API validates the Cognito JWT. If data is needed, the app can use RDS or S3.

B. Deployment flow
Developer → Git push → GitHub Actions → OIDC temporary AWS credentials → Docker build → ECR → new ECS task definition → ECS service deployment → wait for stability → verify Git SHA → verify task count → verify health through CloudFront.

C. Morning recovery flow
07:37 scheduled workflow → check/rebuild with HCP Terraform → deploy application → discover the current CloudFront domain → assume restricted role in personal AWS → UPSERT Route 53 cloud record → verify public HTTPS. 08:17 is a fallback: if the site is already healthy, it skips the duplicate rebuild.

11. Very short memory trick

Remember the platform in seven words:

NETWORK	VPC, subnets, routes, NAT, security groups
CONTAINER	Docker → ECR → ECS Fargate
TRAFFIC	CloudFront → WAF → ALB → target group
DATA	RDS + S3 + Secrets Manager
IDENTITY	IAM/OIDC for deployment; Cognito/JWT for users
AUTOMATION	Terraform/HCP Terraform + GitHub Actions
OBSERVABILITY	CloudWatch logs and metrics

12. One-minute explanation

"LangNation runs inside a private AWS VPC across two Availability Zones. Users enter through Route 53 and CloudFront over HTTPS, and AWS WAF filters malicious requests. CloudFront is the only service allowed to reach the Application Load Balancer. The ALB distributes traffic to ECS Fargate tasks running in private subnets, while RDS MySQL is isolated in private database subnets. S3 provides secure object storage, Secrets Manager protects database credentials, and Cognito provides user authentication with JWT access tokens. Terraform and HCP Terraform manage the infrastructure, while GitHub Actions uses OIDC to deploy containers automatically and recover the disposable sandbox every morning."

Best way to work with AWS
Do not memorize AWS definitions word-for-word. For every service, remember three things: (1) what problem it solves, (2) where it sits in our architecture, and (3) what would break if we removed it.